

"""

Face Aura Bot – Telegram-бот, который анализирует геометрию лица по фото и присылает PDF-отчёт.

КАК ЭТО РАБОТАЕТ (по шагам):

1. Пользователь присылает боту фото лица.
2. MediaPipe находит на фото ~468 ключевых точек лица.
3. По координатам точек считаются простые геометрические соотношения (например: ширина глаза / ширина лица).
4. Каждое соотношение сравнивается с "нормой" (заданной вручную) – получается балл от 0 до 10.
5. Генерируется PDF с фото (с точками), баллами и текстом.
6. PDF отправляется пользователю в чат.

Тебе не нужно ничего менять в этом файле, чтобы запустить бота.

Нужно только вставить свой токен (см. README.md).

"""

```
import os
import io
import math
import logging
from datetime import datetime
```

```
import cv2
import numpy as np
import mediapipe as mp
from PIL import Image, ImageDraw

from fpdf import FPDF

from aiogram import Bot, Dispatcher, F
from aiogram.types import Message,
FSInputFile, BufferedInputFile
from aiogram.filters import CommandStart
import asyncio

logging.basicConfig(level=logging.INFO)

# ==== НАСТРОЙКИ ====
BOT_TOKEN = os.environ.get("BOT_TOKEN",
"ВСТАВЬ_СЮДА_СВОЙ_ТОКЕН")

# ==== ИНИЦИАЛИЗАЦИЯ MEDIAPIPE ====
mp_face_mesh = mp.solutions.face_mesh
face_mesh = mp_face_mesh.FaceMesh(
    static_image_mode=True,
    max_num_faces=1,
    refine_landmarks=True,
    min_detection_confidence=0.5,
)

# Индексы ключевых точек MediaPipe Face
Mesh (468-точечная модель)
LM = {
    "left_eye_outer": 33, "left_eye_inner":
133,
```

```
    "right_eye_outer": 263,  
    "right_eye_inner": 362,  
    "left_eye_top": 159, "left_eye_bottom":  
145,  
    "nose_tip": 1, "nose_bridge": 168,  
    "nose_left": 129, "nose_right": 358,  
    "mouth_left": 61, "mouth_right": 291,  
    "mouth_top": 13, "mouth_bottom": 14,  
    "chin": 152, "forehead": 10,  
    "left_cheek": 234, "right_cheek": 454,  
    "left_jaw": 172, "right_jaw": 397,  
    "left_brow": 105, "right_brow": 334,  
}
```

```
# Метрики: имя -> (функция расчёта,  
норма(среднее), станд.отклонение, описание)  
def dist(p1, p2):  
    return math.hypot(p1[0] - p2[0], p1[1]  
- p2[1])
```

```
def compute_metrics(points):  
    """points – словарь {название_точки:  
(x, y)}"""  
    face_width = dist(points["left_cheek"],  
points["right_cheek"])  
    face_height = dist(points["forehead"],  
points["chin"])  
    eye_w_l =  
dist(points["left_eye_outer"],  
points["left_eye_inner"])  
    mouth_w = dist(points["mouth_left"],  
points["mouth_right"])  
    nose_w = dist(points["nose_left"],
```

```

points["nose_right"])
    jaw_w = dist(points["left_jaw"],
points["right_jaw"])

    metrics_raw = {
        "Симметрия лица": abs(
            dist(points["left_cheek"],
points["nose_tip"])
            - dist(points["right_cheek"],
points["nose_tip"])
        ) / face_width,
        "Пропорции лица (высота/ширина)":
face_height / face_width,
        "Ширина носа / ширина лица": nose_w
/ face_width,
        "Ширина рта / ширина скул": mouth_w
/ face_width,
        "Скулы / челюсть": face_width /
jaw_w,
        "Ширина глаза / ширина лица":
eye_w_l / face_width,
    }
    return metrics_raw

```

```

# Нормы (среднее, стандартное отклонение) –
задай свои на основе
# любой статьи/датасета, который найдёшь.
Пока – примерные значения.
NORMS = {
    "Симметрия лица": (0.02, 0.015),
    "Пропорции лица (высота/ширина)":
(0.90, 0.05),
    "Ширина носа / ширина лица": (0.23,

```

```

0.02),
    "Ширина рта / ширина скул": (0.40,
0.03),
    "Скулы / челюсть": (1.30, 0.10),
    "Ширина глаза / ширина лица": (0.22,
0.015),
}

def score_from_z(z):
    """Переводит отклонение в сигмах в балл
от 0 до 10."""
    score = 10 * math.exp(-0.5 * (z / 2.2)
** 2)
    return round(max(0, min(10, score)), 2)

def analyze_image(image_bgr):
    h, w, _ = image_bgr.shape
    results =
face_mesh.process(cv2.cvtColor(image_bgr,
cv2.COLOR_BGR2RGB))
    if not results.multi_face_landmarks:
        return None, None

    lm =
results.multi_face_landmarks[0].landmark
points = {}
    for name, idx in LM.items():
        points[name] = (lm[idx].x * w,
lm[idx].y * h)

    raw = compute_metrics(points)
    scored = {}

```

```

    for name, value in raw.items():
        mean, std = NORMS[name]
        z = (value - mean) / std
        scored[name] = {
            "value": round(value, 3),
            "norm": mean,
            "z": round(z, 2),
            "score": score_from_z(z),
        }

    overall = round(sum(m["score"] for m in
scored.values()) / len(scored), 2)
    return scored, overall, points

def draw_landmarks_on_image(image_bgr,
points):
    img_rgb = cv2.cvtColor(image_bgr,
cv2.COLOR_BGR2RGB)
    pil_img = Image.fromarray(img_rgb)
    draw = ImageDraw.Draw(pil_img)
    for name, (x, y) in points.items():
        r = 3
        draw.ellipse((x - r, y - r, x + r,
y + r), fill=(255, 60, 60))
    return pil_img

def build_pdf(pil_photo_with_landmarks,
scored, overall, out_path):
    tmp_img_path = out_path.replace(".pdf",
"_photo.jpg")

    pil_photo_with_landmarks.convert("RGB").sav

```

```

e(tmp_img_path, quality=90)

pdf = FPDF()
pdf.add_page()
pdf.set_font("Helvetica", "B", 20)
pdf.cell(0, 15, "Face Aura – отчёт",
ln=True, align="C")

pdf.image(tmp_img_path, x=65, y=25,
w=80)
pdf.ln(95)

pdf.set_font("Helvetica", "B", 16)
pdf.cell(0, 10, f"Общий балл: {overall}
/ 10", ln=True, align="C")
pdf.ln(5)

pdf.set_font("Helvetica", "", 12)
for name, data in scored.items():
    pdf.set_font("Helvetica", "B", 12)
    pdf.cell(0, 8, f"{name}:
{data['score']} / 10", ln=True)
    pdf.set_font("Helvetica", "", 10)
    pdf.cell(
        0, 6,
        f"    Показатель:
{data['value']}    Норма: {data['norm']}
Отклонение: {data['z']}σ",
        ln=True,
    )
    pdf.ln(2)

pdf.output(out_path)
os.remove(tmp_img_path)

```

```

# ==== TELEGRAM BOT ====
bot = Bot(token=BOT_TOKEN)
dp = Dispatcher()

@dp.message(CommandStart())
async def start_handler(message: Message):
    await message.answer(
        "Привет! Пришли мне фото лица
(анфас, хорошее освещение), "
        "и я пришлю PDF-отчёт с анализом
геометрии лица."
    )

@dp.message(F.photo)
async def photo_handler(message: Message):
    await message.answer("Анализирую
фото...")

    photo = message.photo[-1]
    file = await
bot.get_file(photo.file_id)
    file_bytes = await
bot.download_file(file.file_path)

    npimg =
np.frombuffer(file_bytes.read(), np.uint8)
    image_bgr = cv2.imdecode(npimg,
cv2.IMREAD_COLOR)

    result = analyze_image(image_bgr)

```



```
        if result[0] is None:
            await message.answer("Не удалось  
найти лицо на фото. Пришли другое фото  
анфас.")
            return

        scored, overall, points = result
        pil_photo =
draw_landmarks_on_image(image_bgr, points)

        out_path =
f"/tmp/report_{message.from_user.id}_{int(d  
atetime.now().timestamp())}.pdf"
        build_pdf(pil_photo, scored, overall,
out_path)

        doc = FSInputFile(out_path)
        await message.answer_document(doc,
caption=f"Готово! Общий балл:  
{overall}/10")
        os.remove(out_path)

async def main():
    await dp.start_polling(bot)

if __name__ == "__main__":
    asyncio.run(main())
```